



نام دانشجو :

رضا ندیمی

شماره دانشجویی:

832204022

دانشکده سامانه های هوشمند - مکترونیک

تمرینات مکترونیک 2 سری سوم

تاریخ:

1405/02/14

استاد مربوطه:

دکتر تارویردی زاده

فضای توسعه ی مورد استفاده با توجه به عدم دسترسی به مخازن نرم افزار بین المللی	2
1. یک عدد button-push به پایه وقفه خارجی میکروکنترلر متصل کرده و هر بار که آن را فشار می‌دهید یک شمارنده در حال شمارش بوده و یک واحد (فقط یک واحد)، افزایش یابد (با هر بار فشار دادن کلید). تعداد فشار دادن های کلید را روی LCD نمایش دهید. یک عدد button-push به پای ه وقفه خارجی دیگر میکروکنترلر متصل کنید بطوریکه هر بار که این کلید فشار داده می شود، شمارنده فوق را ریست کند	3
(i) بخش STM32CubeMX	3
(ii) بخش Keil	5
(iii) بخش شبیه سازی Proteus	7
2. یک میز مسابقه سه نفره را به گونه ای طراحی کنید که هرگاه هر کدام از button-push های P1 یا P2 یا P3 زودتر فشار داده شود LED مربوط به آن روشن میشود و LED های دیگر روشن نمیشوند. با آزاد بودن همه button-push های P1 یا P2 یا P3، سیستم در وضعیت آماده به کار قرار گیرد.....	9
(i) بخش STM32CubeMX	9
(ii) بخش Keil	11
(iii) بخش شبیه سازی Proteus	13
3.	14
(i) بخش STM32CubeMX	14
(ii) بخش Keil	16
(iii) بخش شبیه سازی Proteus	21
4. برخی از انکودرهایی که برای سنجش سرعت موتور و یا اندازه‌گیری میزان دوران موتور مورد استفاده قرار می گیرند، پالس هایی بصورت مقابل تولید میکنند. با استفاده از مولد پالس (سیگنال ژنراتور)، سیگنال هایی بصورت مقابل و با فرکانس 1 kHz تولید کرده و میزان دوران موتور را اندازه بگیرید و روی LCD نمایش دهید. فرض کنید انکودر شما در هر دور از دوران موتور، 720 عدد پالس تولید میکند. جهت دوران موتور را هم تشخیص دهید. برای تولید پالس، میتوانید از خود میکروکنترلر نیز استفاده کنید	23
(i) بخش STM32CubeMX	23
(ii) بخش Keil	25
(iii) بخش شبیه سازی Proteus	29

فضای توسعه ی مورد استفاده با توجه به عدم دسترسی به مخازن نرم افزار بین المللی

1. برای STM32 CubeMX پک F1 نسخه ی V1.8.0

2. برای Keil پک Keil.STM32F1xx_DFP.2.4.0

3. برای Keil پک ARM.CMSIS.4.5.0 متعلق به سال 2015 (آخرین نسخه ای که در نت ملی توانستم

پیدا کنم. الان سال 2026 می باشد یعنی چیزی حدود 11 سال پیش)

1. یک عدد button-push به پایه وقفه خارجی میکروکنترلر متصل کرده و هر بار که آن را فشار می‌دهید یک شمارنده در حال شمارش بوده و یک واحد (فقط یک واحد)، افزایش یابد (با هر بار فشار دادن کلید). تعداد فشار دادن های کلید را روی LCD نمایش دهید. یک عدد button-push به پایه وقفه خارجی دیگر میکروکنترلر متصل کنید بطوریکه هر بار که این کلید فشار داده می‌شود، شمارنده فوق را ریست کند

(i) بخش STM32CubeMX

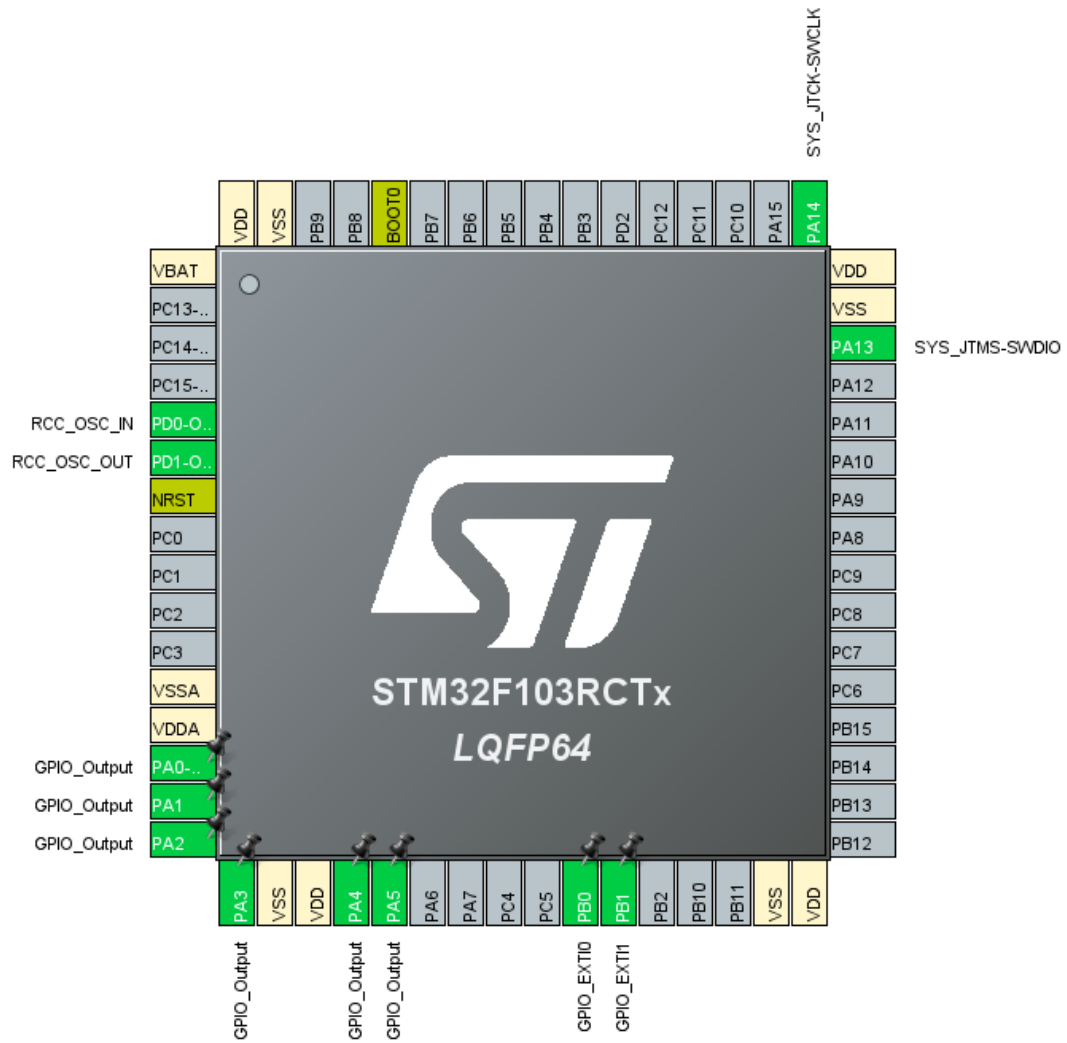
میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

پایه های 0 تا 5 پورت A را به صورت Output برای نمایگر و پایه ای 0 و 1 پورت B را به صورت اینترپت خارجی تنظیم میکنیم. و در NVIC اینترپت ان ها رو enable می کنیم.

GPIO RCC SYS NVIC							
Search Signals Search (Ctrl+F) Show only Modified Pins							
Pin Na...	Signal on ...	GPIO outp...	GPIO mode	GPIO Pull...	Maximum ...	User Label	Modified
PA0-WKUP	n/a	Low	Output Pus...	No pull-up ...	Low		☐
PA1	n/a	Low	Output Pus...	No pull-up ...	Low		☐
PA2	n/a	Low	Output Pus...	No pull-up ...	Low		☐
PA3	n/a	Low	Output Pus...	No pull-up ...	Low		☐
PA4	n/a	Low	Output Pus...	No pull-up ...	Low		☐
PA5	n/a	Low	Output Pus...	No pull-up ...	Low		☐
PB0	n/a	n/a	External Int...	Pull-down	n/a		☒
PB1	n/a	n/a	External Int...	Pull-down	n/a		☒

اینترپت ها به صورت rising edge trigger می باشند.

پروگرمر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم



Keil بخش (ii)

کتابخانه های Lcd و string رو ادد می کنیم

```
/* USER CODE BEGIN Includes */
#include "lcd.h"
#include <string.h>
```

پین ها رو مشخص میکنیم.

```
/* USER CODE BEGIN PTD */
#define RS_PIN GPIO_PIN_0
#define RS_PORT GPIOA

#define ENB_PIN GPIO_PIN_1
#define ENB_PORT GPIOA

#define LCD_PORTS {GPIOA,GPIOA,GPIOA,GPIOA}
#define LCD_PINS {GPIO_PIN_2,GPIO_PIN_3,GPIO_PIN_4,GPIO_PIN_5}
```

شمارنده خودمون رو معرفی و مقدار اولیه می دهیم.

```
/* USER CODE BEGIN PV */
uint8_t counter=0;
```

با استفاده از کالک اینتراپت شمارنده خود را افزایش و یا ریست می کنیم.

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    if(GPIO_Pin == GPIO_PIN_0){
        counter++;
    }
    if(GPIO_Pin == GPIO_PIN_1){
        counter =0;
    }
}
```

در داخل main

Lcd خود را معرفی و تنظیم میکنیم.

```
Lcd_PortType Ports[] = LCD_PORTS;
Lcd_PinType Pins[] = LCD_PINS;

Lcd_HandleTypeDef lcd;
lcd = Lcd_create(Ports, Pins, RS_PORT, RS_PIN, ENB_PORT, ENB_PIN,
LCD_4_BIT_MODE);
char buff[20];

Lcd_clear(&lcd);
Lcd_cursor(&lcd,0,0);
Lcd_string(&lcd,"Counter: ");
```

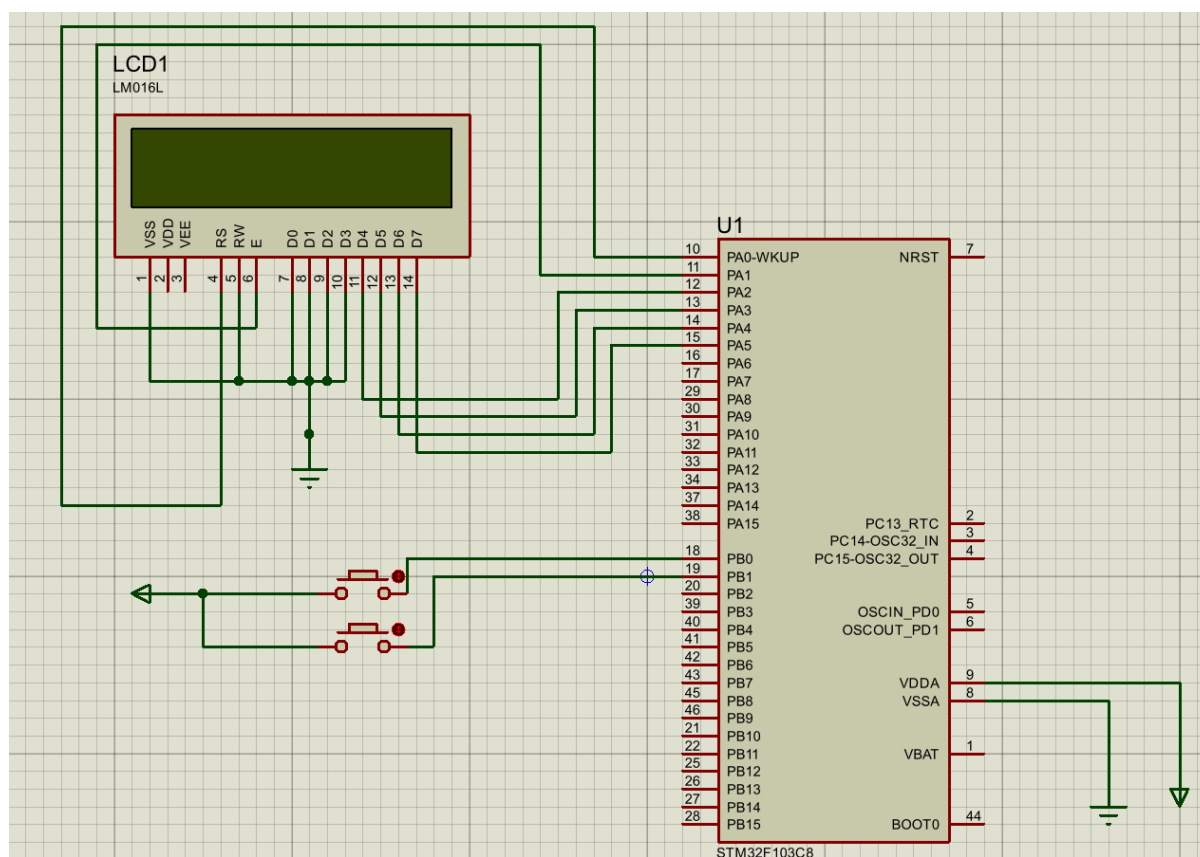
در داخل while

یک عدد سه رقمی رزرو میکنیم و در داخل آن مقدار شمارنده خود را می نویسیم.

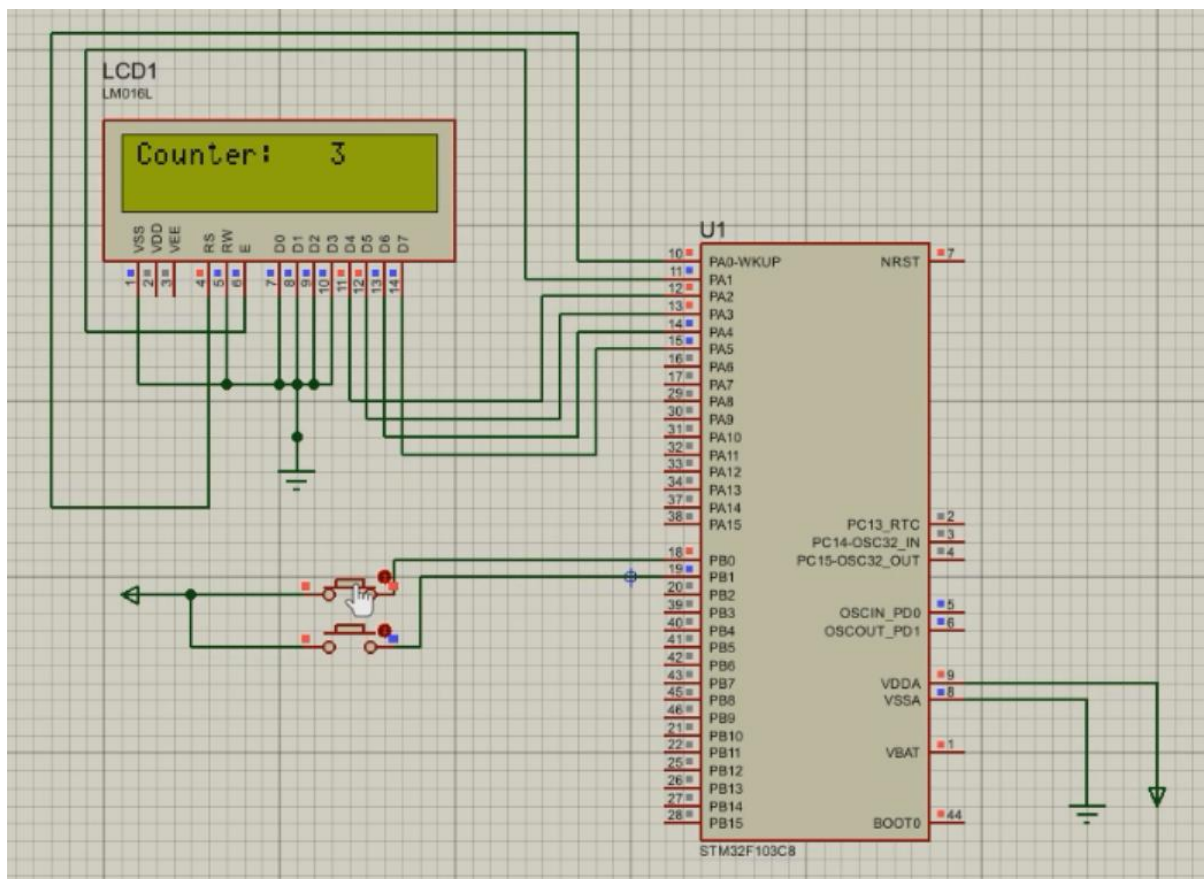
```
sprintf(buff, "%3d", counter);
Lcd_cursor(&lcd,0,9);
Lcd_string(&lcd,buff);

HAL_Delay(500);
```

(iii) بخش شبیه سازی Proteus



LCD (LM016L) و پوش باتن ها و میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهم و به شکل بالا کدار رو میبندیم در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.



ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

2. یک میز مسابقه سه نفره را به گونه ای طراحی کنید که هرگاه هر کدام از button-push های P1 یا P2 یا P3 زودتر فشار داده شود LED مربوط به آن روشن میشود و LED های دیگر روشن نمیشوند. با آزاد بودن همه button-push های P1 یا P2 یا P3، سیستم در وضعیت آماده به کار قرار گیرد

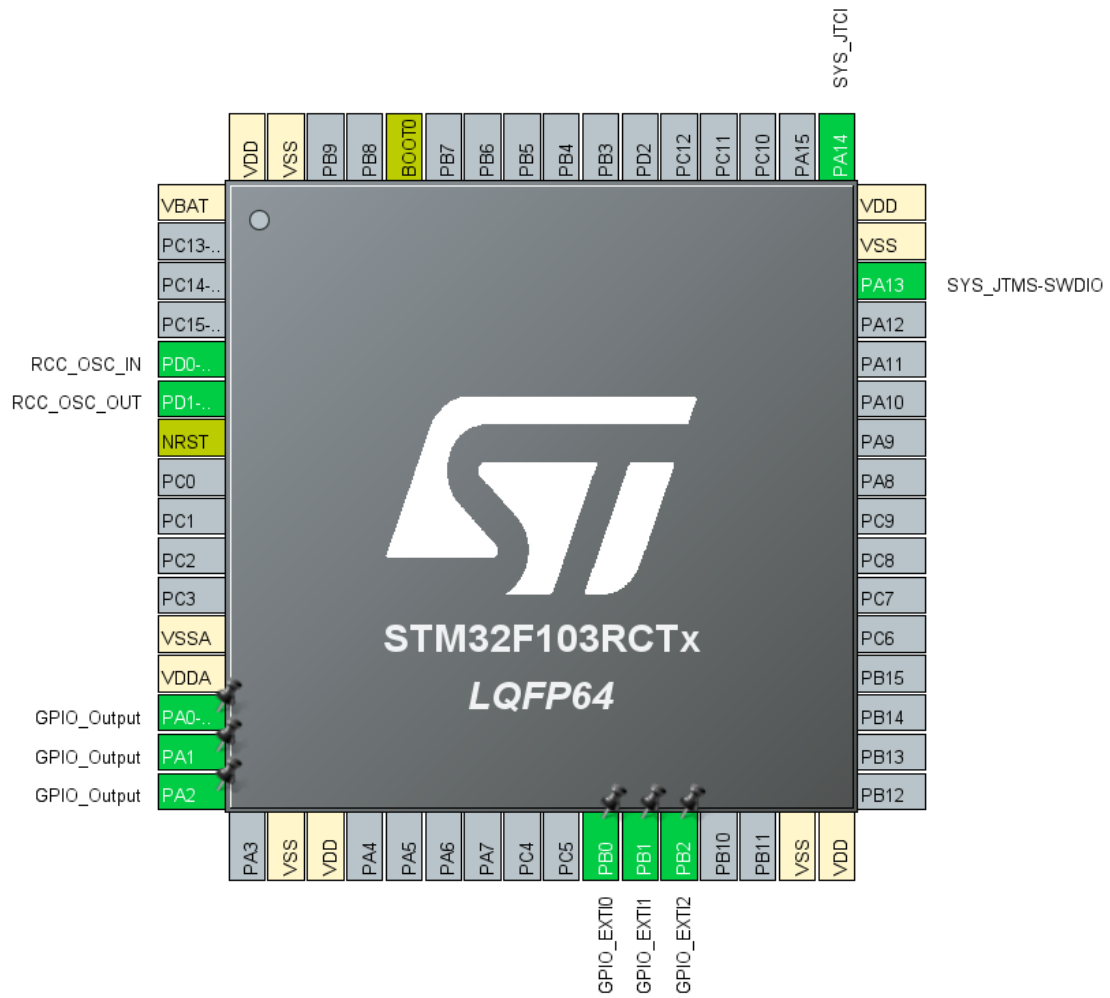
(i) بخش STM32CubeMX

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

پایه 0 تا 2 پورت A را به عنوان OUTPUT

پایه های 0 تا 2 پورت B را به عنوان اکسترنال اینترپت با Pull-up و Falling edge trigger تنظیم میکنیم

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم



پیکربندی پین های خود را مشخص می کنیم

```
/* USER CODE BEGIN PD */
#define BUTTON1_PIN  GPIO_PIN_0
#define BUTTON2_PIN  GPIO_PIN_1
#define BUTTON3_PIN  GPIO_PIN_2
#define BUTTON_PORT  GPIOB

#define LED1_PIN      GPIO_PIN_0
#define LED2_PIN      GPIO_PIN_1
#define LED3_PIN      GPIO_PIN_2
#define LED_PORT      GPIOA
```

متغیر برنده بودن رو به صورت volatile معرفی می کنیم تا کاپایر از تغییرات ان در خارج از main چشم پوشی نکند.

```
volatile uint8_t winner = 0; // 0 = no winner, 1/2/3 = winning player
```

در داخل کالک منطق بازی رو پیاده سازی می کنیم

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (winner == 0)
    {
        if (GPIO_Pin == BUTTON1_PIN)
        {
            winner = 1;
            HAL_GPIO_WritePin(LED_PORT, LED1_PIN, GPIO_PIN_SET);
            HAL_GPIO_WritePin(LED_PORT, LED2_PIN | LED3_PIN, GPIO_PIN_RESET);
        }
        else if (GPIO_Pin == BUTTON2_PIN)
        {
            winner = 2;
            HAL_GPIO_WritePin(LED_PORT, LED2_PIN, GPIO_PIN_SET);
            HAL_GPIO_WritePin(LED_PORT, LED1_PIN | LED3_PIN, GPIO_PIN_RESET);
        }
        else if (GPIO_Pin == BUTTON3_PIN)
        {
            winner = 3;
            HAL_GPIO_WritePin(LED_PORT, LED3_PIN, GPIO_PIN_SET);
            HAL_GPIO_WritePin(LED_PORT, LED1_PIN | LED2_PIN, GPIO_PIN_RESET);
        }
    }
}
```

در داخل main همه ی پین ها رو به حالت ریست میبریم

```
/* USER CODE BEGIN 2 */
    HAL_GPIO_WritePin(LED_PORT, LED1_PIN | LED2_PIN | LED3_PIN,
GPIO_PIN_RESET);
```

و در داخل while

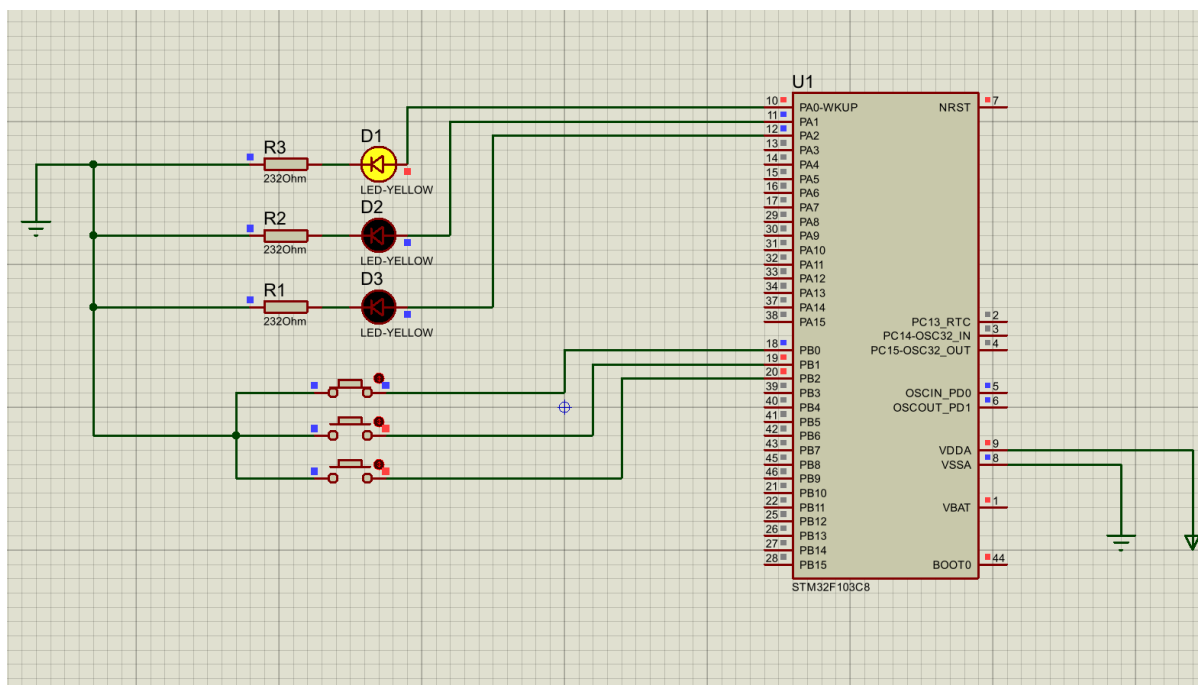
مقادیر کلید ها را می خوانیم و در صورت نیاز پین ها را ریست می کنیم.

```
/* USER CODE BEGIN 3 */
    if (winner != 0)
    {
        uint8_t btn1 = HAL_GPIO_ReadPin(BUTTON_PORT, BUTTON1_PIN);
        uint8_t btn2 = HAL_GPIO_ReadPin(BUTTON_PORT, BUTTON2_PIN);
        uint8_t btn3 = HAL_GPIO_ReadPin(BUTTON_PORT, BUTTON3_PIN);

        if (btn1 == GPIO_PIN_SET && btn2 == GPIO_PIN_SET && btn3 ==
GPIO_PIN_SET)
        {
            winner = 0;
            HAL_GPIO_WritePin(LED_PORT, LED1_PIN | LED2_PIN | LED3_PIN,
GPIO_PIN_RESET);
        }
    }
    HAL_Delay(10);
```

(iii) بخش شبیه سازی Proteus

3 LED و 3 مقاومت 232 اهمی و میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهم و به شکل زیر مدار رو میبندیم در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.



ویدیوی شبیه سازی در داخل پوشه تمرین مربوطه موجود می باشد.

طراحی و پیاده‌سازی سیستم تشخیص تداخل چندگانه با استفاده از EXTI و GPIO.

یک سیستم نظارتی را در نظر بگیرید که دارای سه حسگر (مانند دکمه‌های فشاری) متصل به سه پین ورودی متفاوت میکروکنترلر STM32F103xx باشد. هدف شما طراحی سیستمی است که:

هر حسگر با فشار داده شدن، یک وقفه تولید کند و شماره حسگر فعال شده را روی LCD نشان دهد.

اگر دو یا سه حسگر به صورت همزمان فشار داده شوند، باید پیام هشدار "تداخل چندگانه (Multiple Interference)" (یا پیغام مشابه)، روی LCD نشان داده شود.

با رها کردن هر یک از حسگرها، سیستم باید به حالت اولیه بازگردد و روی LCD، عبارت Ready نمایش داده شود.

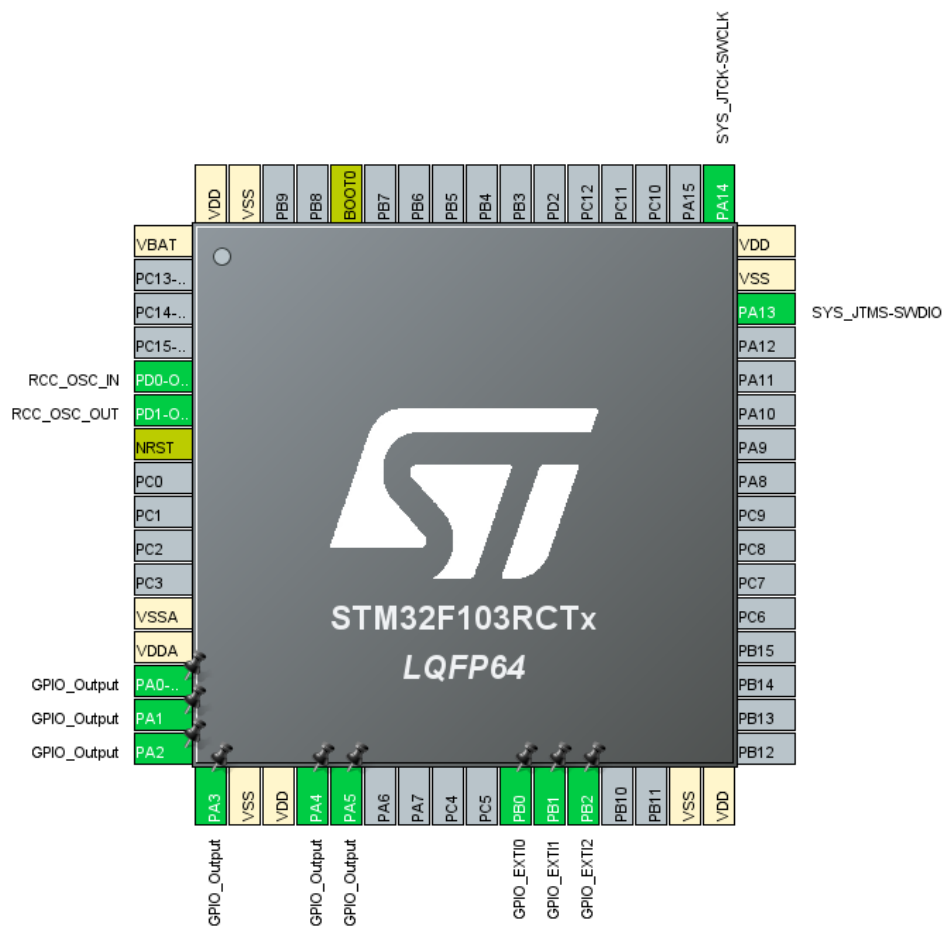
(i) بخش STM32CubeMX

میکروکنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

پایه 0 تا 5 پورت A رو به حالت GPIO Output قرار میدهم

پایه 0 تا 2 پورت B رو به حالت اکسترنال اینتراپت با Pull-up و rising and faling edge trigger قرار میدهم

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم



کتابخانه های lcd و string و stdio رو ادد می کنیم

```
/* USER CODE BEGIN Includes */
#include "lcd.h"
#include <string.h>
#include <stdio.h>
```

پین های مورد استفاده در lcd و push button ها رو مشخص می کنیم.

```
/* USER CODE BEGIN PD */
// Button pins (PB0..2)
#define BTN1_PIN    GPIO_PIN_0
#define BTN2_PIN    GPIO_PIN_1
#define BTN3_PIN    GPIO_PIN_2
#define BTN_PORT    GPIOB

// LCD control
#define RS_PIN      GPIO_PIN_0
#define RS_PORT     GPIOA
#define ENB_PIN     GPIO_PIN_1
#define ENB_PORT    GPIOA
#define LCD_PORTS   {GPIOA, GPIOA, GPIOA, GPIOA}
#define LCD_PINS    {GPIO_PIN_2, GPIO_PIN_3, GPIO_PIN_4, GPIO_PIN_5}
```

متغیر های فعال شدن button ها و چند تا کلید همزمان فشرده شدن رو به صورت volatile معرفی می کنیم تا کامپایلر از تغییرات ان ها در خارج از main مطلع شود و انها را در نظر بگیرد

```
/* USER CODE BEGIN PV */
Lcd_HandleTypeDef lcd;

volatile uint8_t btn1_flag = 0;
volatile uint8_t btn2_flag = 0;
volatile uint8_t btn3_flag = 0;

volatile uint8_t button_event_occurred = 0;

volatile uint8_t was_multi = 0;
```

منطق برنامه رو در callback می نویسیم و با استفاده از یک switch احتمال های مختلف رو بررسی میکنیم.

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    static uint32_t last_time[3] = {0}; // debounce timers for PB0..PB2
    uint32_t now = HAL_GetTick();
    GPIO_PinState state;

    switch (GPIO_Pin)
    {
        case BTN1_PIN:
            if (now - last_time[0] > 50) // 50 ms debounce
            {
                last_time[0] = now;
                state = HAL_GPIO_ReadPin(BTN_PORT, BTN1_PIN);
                if (state == GPIO_PIN_RESET) // active low → pressed
                {
                    if (btn1_flag == 0)
                    {
                        btn1_flag = 1;
                        button_event_occurred = 1;
                    }
                }
                else // released
                {
                    if (btn1_flag == 1)
                    {
                        btn1_flag = 0;
                        button_event_occurred = 1;
                    }
                }
            }
            break;

        case BTN2_PIN:
            if (now - last_time[1] > 50)
            {
                last_time[1] = now;
                state = HAL_GPIO_ReadPin(BTN_PORT, BTN2_PIN);
                if (state == GPIO_PIN_RESET)
                {
                    if (btn2_flag == 0)
                    {
                        btn2_flag = 1;
                        button_event_occurred = 1;
                    }
                }
            }
        }
    }
```

```

        else
        {
            if (btn2_flag == 1)
            {
                btn2_flag = 0;
                button_event_occurred = 1;
            }
        }
    }
    break;

case BTN3_PIN:
    if (now - last_time[2] > 50)
    {
        last_time[2] = now;
        state = HAL_GPIO_ReadPin(BTN_PORT, BTN3_PIN);
        if (state == GPIO_PIN_RESET)
        {
            if (btn3_flag == 0)
            {
                btn3_flag = 1;
                button_event_occurred = 1;
            }
        }
        else
        {
            if (btn3_flag == 1)
            {
                btn3_flag = 0;
                button_event_occurred = 1;
            }
        }
    }
    break;

default:
    break;
}
}

```

در داخل main

Lcd رو معرفی و تنظیمات اولیه رو روش انجام میدیم مثلا نمایش ready در ابتدا و همچنین متغیری برای نگهداشتن کلید قبلی فشرده شده رو هم در اینجا معرفی میکنیم.

```
/* USER CODE BEGIN 2 */

Lcd_PortType ports[] = LCD_PORTS;
Lcd_PinType pins[] = LCD_PINS;
lcd = Lcd_create(ports, pins, RS_PORT, RS_PIN, ENB_PORT, ENB_PIN,
LCD_4_BIT_MODE);
Lcd_clear(&lcd);

Lcd_cursor(&lcd, 0, 0);
Lcd_string(&lcd, "Ready", 0, 0);
Lcd_cursor(&lcd, 1, 0);
Lcd_string(&lcd, " ", 1, 0);

uint8_t prev_count = 0;
uint8_t prev_btn1 = 0, prev_btn2 = 0, prev_btn3 = 0;
```

در داخل while کلید های فشرده شده رو بررسی میکنیم و با توجه به کلید های فشرده شده LCD رو اپدیت می کنیم.

```
if (button_event_occurred ||
    btn1_flag != prev_btn1 ||
    btn2_flag != prev_btn2 ||
    btn3_flag != prev_btn3)
{
    button_event_occurred = 0;

    uint8_t cnt = btn1_flag + btn2_flag + btn3_flag;

    uint8_t single_sensor = 0;
    if (cnt == 1)
    {
        if (btn1_flag) single_sensor = 1;
        else if (btn2_flag) single_sensor = 2;
        else single_sensor = 3;
    }

    if (cnt >= 2)
        was_multi = 1;
    else if (cnt == 0)
```

```

        was_multi = 0;

char lcd_line1[17] = "";
char lcd_line2[17] = "";

if (cnt >= 2)
{
    strcpy(lcd_line1, "Multiple Interf");
    strcpy(lcd_line2, "erence");
}
else if (cnt == 0)
{
    strcpy(lcd_line1, "Ready");
    strcpy(lcd_line2, "");
}
else
{
    if (was_multi)
    {
        strcpy(lcd_line1, "Ready");
        strcpy(lcd_line2, "");
    }
    else
    {
        sprintf(lcd_line1, "Sensor %d", single_sensor);
        strcpy(lcd_line2, "");
    }
}

Lcd_clear(&lcd);
Lcd_cursor(&lcd, 0, 0);
Lcd_string(&lcd, lcd_line1);
Lcd_cursor(&lcd, 1, 0);
Lcd_string(&lcd, lcd_line2);

// Remember previous
prev_count = cnt;
prev_btn1 = btn1_flag;
prev_btn2 = btn2_flag;
prev_btn3 = btn3_flag;
}

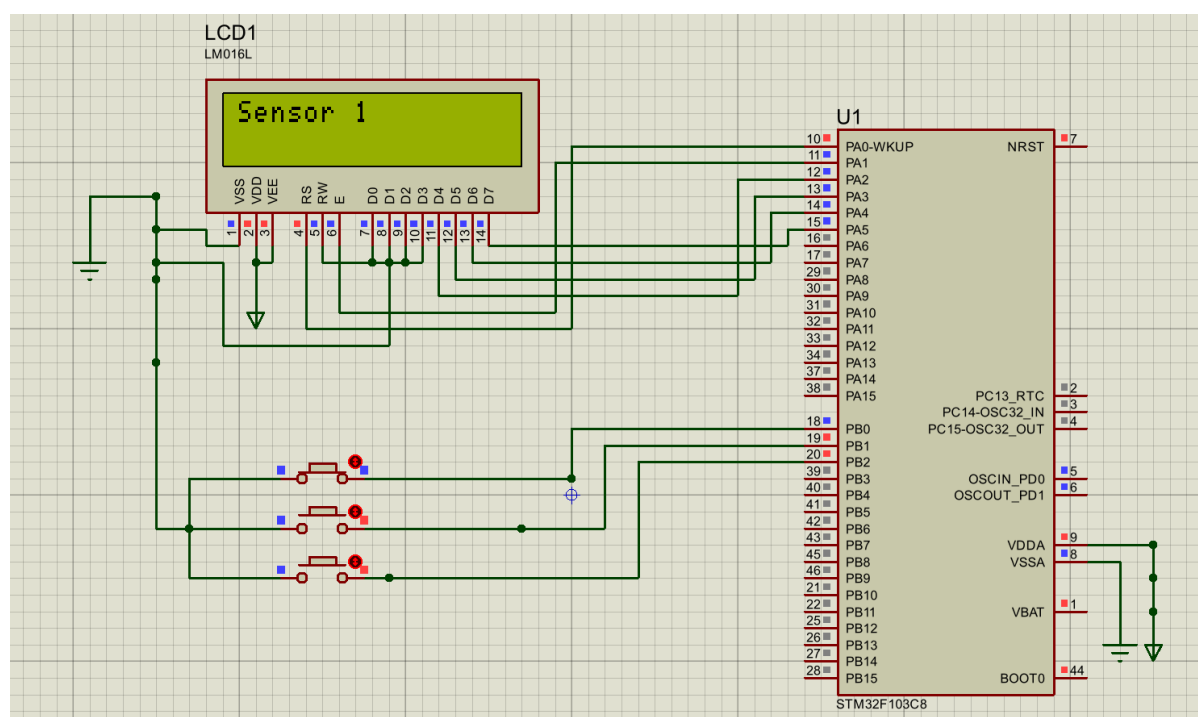
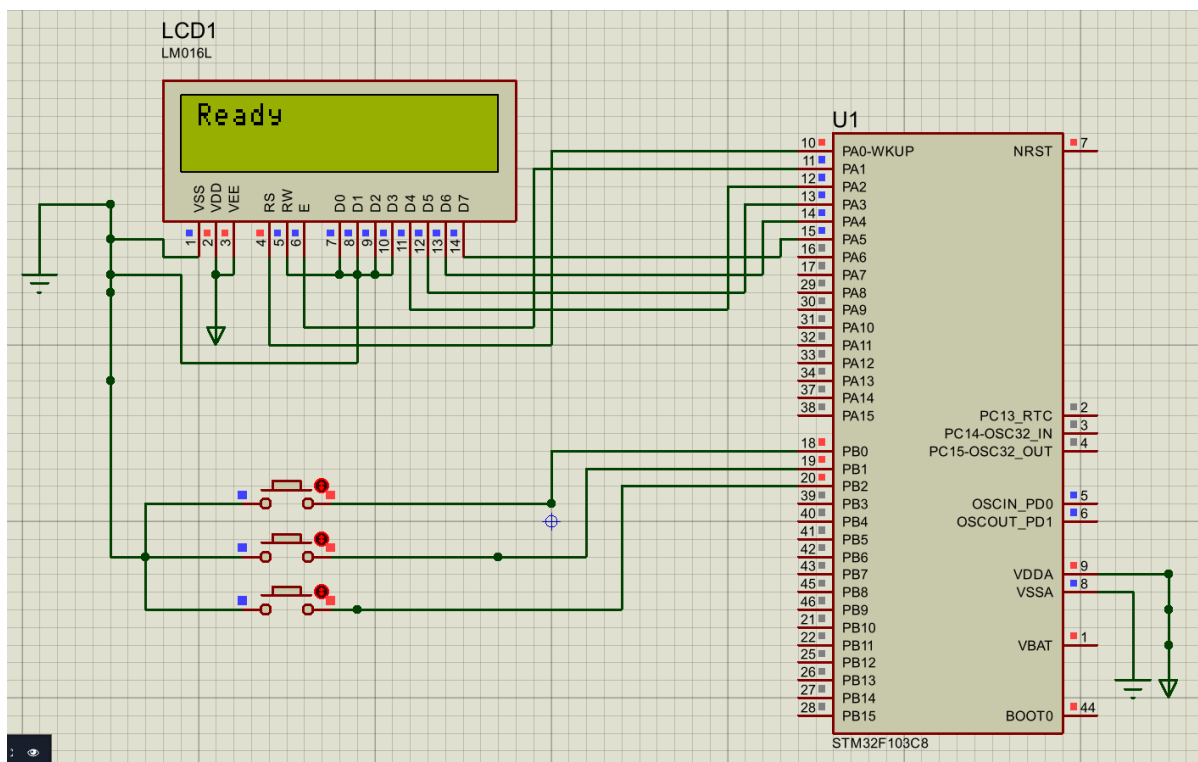
HAL_Delay(10);

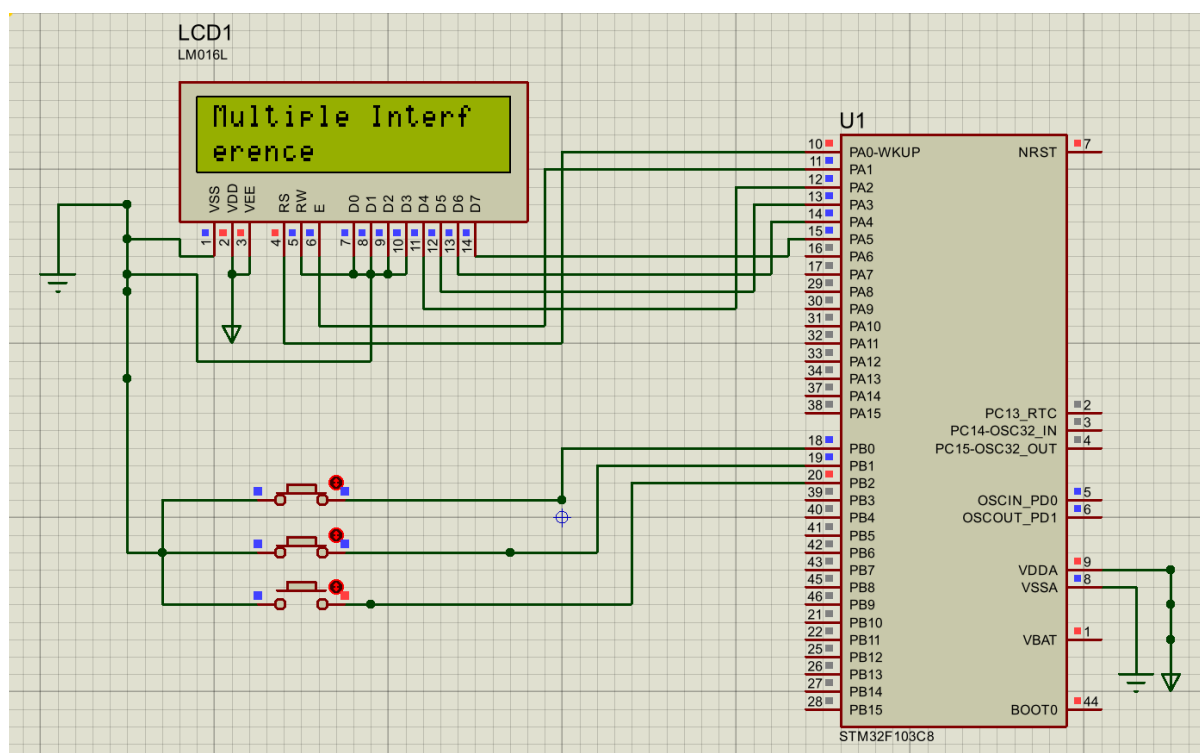
```

(iii) بخش شبیه سازی Proteus

میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) و پوش باتن ها و LM016L LCD رو در محیط شبیه سازی قرار میدهم و مدار را مانند شکل زیر می بندیم.

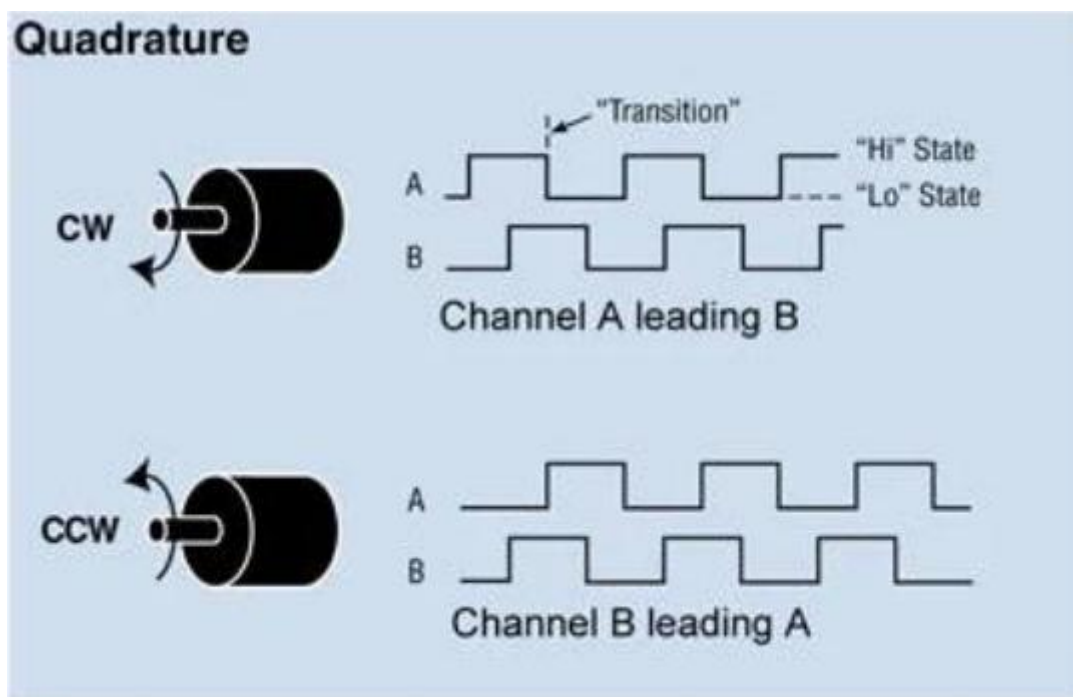
در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.





ویدیو ی شبیه سازی در پوشه تمرین مورد نظر موجود می باشد

4. برخی از انکودرهایی که برای سنجش سرعت موتور و یا اندازه‌گیری میزان دوران موتور مورد استفاده قرار می‌گیرند، پالس‌هایی بصورت مقابل تولید میکنند. با استفاده از مولد پالس (سیگنال ژنراتور)، سیگنال‌هایی بصورت مقابل و با فرکانس 1 kHz تولید کرده و میزان دوران موتور را اندازه بگیرید و روی LCD نمایش دهید. فرض کنید انکودر شما در هر دور از دوران موتور، 720 عدد پالس تولید میکند. جهت دوران موتور را هم تشخیص دهید. برای تولید پالس، میتوانید از خود میکروکنترلر نیز استفاده کنید



(i) بخش STM32CubeMX

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

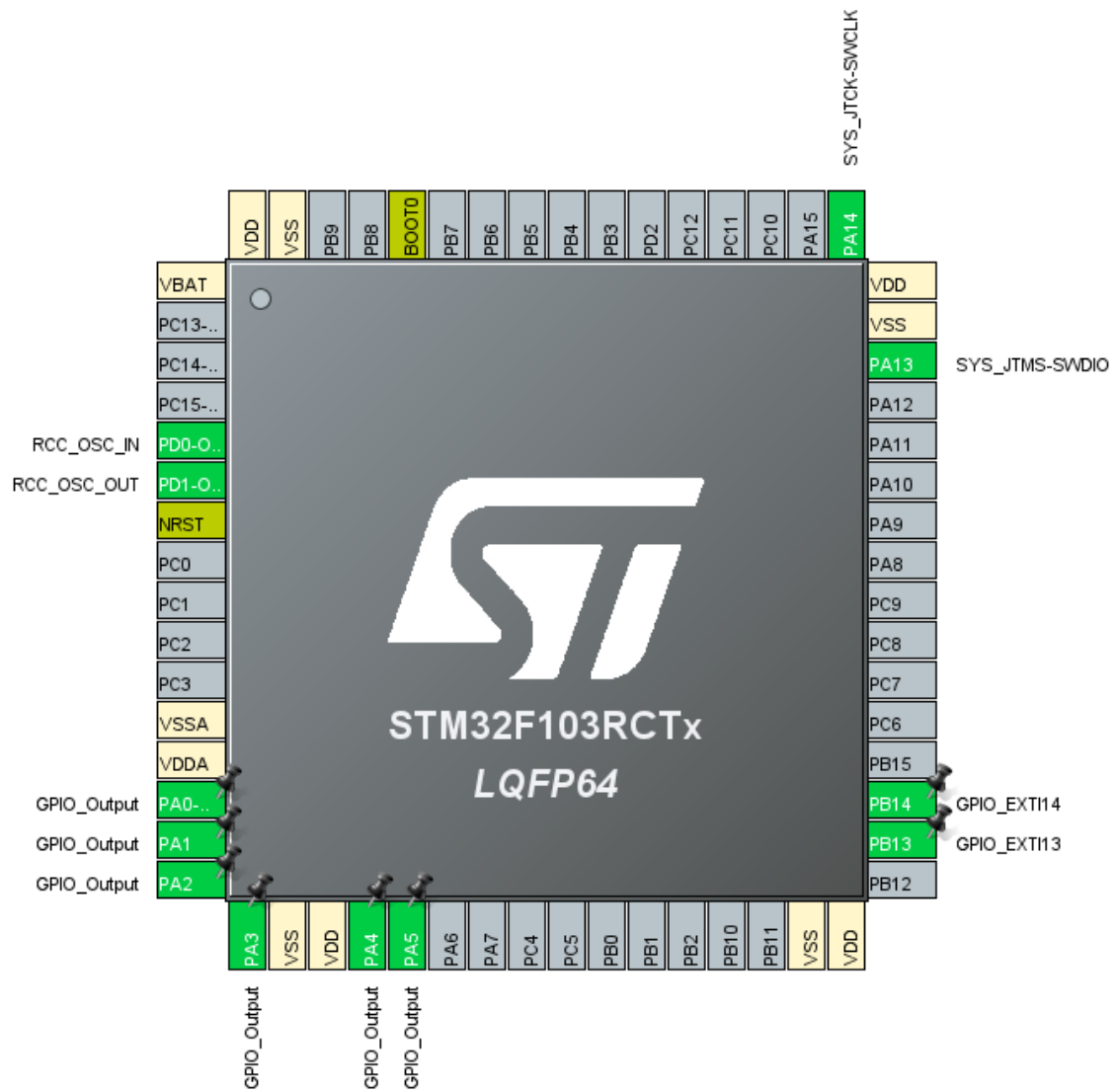
پایه 0 تا 5 پورت A رو به حالت GPIO Output قرار میدهیم (برای LCD)

پایه های 14 و 13 پورت B رو به حالت اکسترنال اینترپت با Pull-up و rising edge trigger

قرار می‌دهیم.

در بخش NVIC اکسترنال اینترپت پایه های 10 تا 15 را فعال می‌کنیم

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم



Keil بخش (ii)

کتابخانه های مربوط به LCD ,STDIO و String رو ایمپورت می کنیم.

```
/* USER CODE BEGIN Includes */
#include "lcd.h"
#include <stdio.h>
#include <string.h>
```

پیکربندی پایه ها و سرعت اینکودر رو تعیین می کنیم.

```
/* USER CODE BEGIN PD */
#define ENC_A_PIN      GPIO_PIN_13
#define ENC_B_PIN      GPIO_PIN_14
#define ENC_PORT        GPIOB

#define RS_PIN          GPIO_PIN_0
#define RS_PORT          GPIOA
#define EN_PIN          GPIO_PIN_1
#define EN_PORT          GPIOA
#define LCD_DATA_PORTS {GPIOA, GPIOA, GPIOA, GPIOA}
#define LCD_DATA_PINS  {GPIO_PIN_2, GPIO_PIN_3, GPIO_PIN_4, GPIO_PIN_5}

#define PULSES_PER_REV  720
```

شمارنده های خود را به صورت Volatile معرفی می کنیم.

```
/* USER CODE BEGIN PV */
Lcd_HandleTypeDef lcd;

volatile uint32_t pulse_counter = 0;
volatile int8_t direction      = 0;    // 1 = FWD, -1 = REV
```

تابع کلید

```
uint8_t Is_Stable_High(GPIO_TypeDef *port, uint16_t pin)
{
    for (volatile int i = 0; i < 25; i++);
    return (HAL_GPIO_ReadPin(port, pin) == GPIO_PIN_SET);
}
```

این تابع با یک تأخیر کوتاه نرم‌افزاری، سطح پایدار پین را می‌خواند و در صورت بالا بودن (High) مقدار 1 را برمی‌گرداند.

تابع کالیک

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == ENC_A_PIN)
    {
        // Channel A rising edge → always count one pulse
        pulse_counter++;

        // Direction from B
        if (HAL_GPIO_ReadPin(ENC_PORT, ENC_B_PIN) == GPIO_PIN_RESET)
            direction = 1;
        else
            direction = -1;
    }
    else if (GPIO_Pin == ENC_B_PIN)
    {
        // Channel B rising edge → only update direction
        if (HAL_GPIO_ReadPin(ENC_PORT, ENC_A_PIN) == GPIO_PIN_SET)
            direction = 1;
        else
            direction = -1;
    }
}
```

این تابع وقفه با تشخیص لبه‌های بالارونده کانال‌های A و B اینکودر، پالس‌ها را می‌شمارد و جهت چرخش را مشخص می‌کند.

تابع lcd

```
void LCD_Update(void)
{
    uint32_t cnt = pulse_counter;
    double rnd = (double)cnt / PULSES_PER_REV;

    char buf1[16], buf2[16];
    const char *dir = (direction == 1) ? "FWD" : (direction == -1) ? "REV" :
    "---";

    sprintf(buf1, "revol: %.2f", rnd);
    sprintf(buf2, "Counter: %lu", cnt);
}
```

```

    Lcd_clear(&lcd);
    Lcd_cursor(&lcd, 0, 0);
    Lcd_string(&lcd, buf1);
    Lcd_cursor(&lcd, 0, 11);
    Lcd_string(&lcd, dir);
    Lcd_cursor(&lcd, 1, 0);
    Lcd_string(&lcd, buf2);
}

```

این تابع تعداد دور و شمارنده پالس را محاسبه کرده و همراه با جهت چرخش روی LCD نمایش می‌دهد.

داخل main:

```

Lcd_PortType ports[] = LCD_DATA_PORTS;
Lcd_PinType pins[] = LCD_DATA_PINS;
lcd = Lcd_create(ports, pins, RS_PORT, RS_PIN, EN_PORT, EN_PIN,
LCD_4_BIT_MODE);
Lcd_clear(&lcd);

/* Show initial screen */
LCD_Update();

```

در این بخش LCD با پین‌های تعریف‌شده راه‌اندازی شده و صفحه اولیه با فراخوانی LCD_Update نمایش داده می‌شود.

داخل while

```

/* USER CODE BEGIN 3 */
static uint32_t last_cnt = 0;
if (pulse_counter != last_cnt)
{
    last_cnt = pulse_counter;
    LCD_Update();
}
HAL_Delay(200);

```

حلقه اصلی با مقایسه شمارنده فعلی و قبلی، فقط در صورت تغییر، نمایشگر را به‌روز کرده و سپس 200 میلی‌ثانیه مکث می‌کند.

دو متغیر `pulse_counter` و `direction` برای ذخیره تعداد پالس‌ها و جهت چرخش تعریف می‌شوند و تابع وقفه‌ای `HAL_GPIO_EXTI_Callback` با هر لبه سیگنال کانال A یا B، شمارنده را افزایش داده و جهت را بر اساس وضعیت کانال دیگر تشخیص می‌دهد.

تابع `LCD_Update` با محاسبه تعداد دور $(\text{counter} / 720)$ و جهت، رشته‌های مربوطه را با `sprintf` ساخته و روی دو سطر LCD نمایش می‌دهد؛ در پایان تابع `main`، ساختار LCD مقداردهی اولیه شده و با فراخوانی `LCD_Update` صفحه آغازین نشان داده می‌شود.

داخل حلقه بینهایت، یک متغیر ایستا به نام `last_cnt` نگهداری می‌شود که آخرین مقدار خوانده شده از `pulse_counter` را به خاطر می‌سپارد.

اگر مقدار جدید با `last_cnt` تفاوت داشته باشد (یعنی اینکودر جابه‌جا شده باشد)، این متغیر به‌روزرسانی شده و تابع `LCD_Update` برای نمایش دوباره فراخوانی می‌شود.

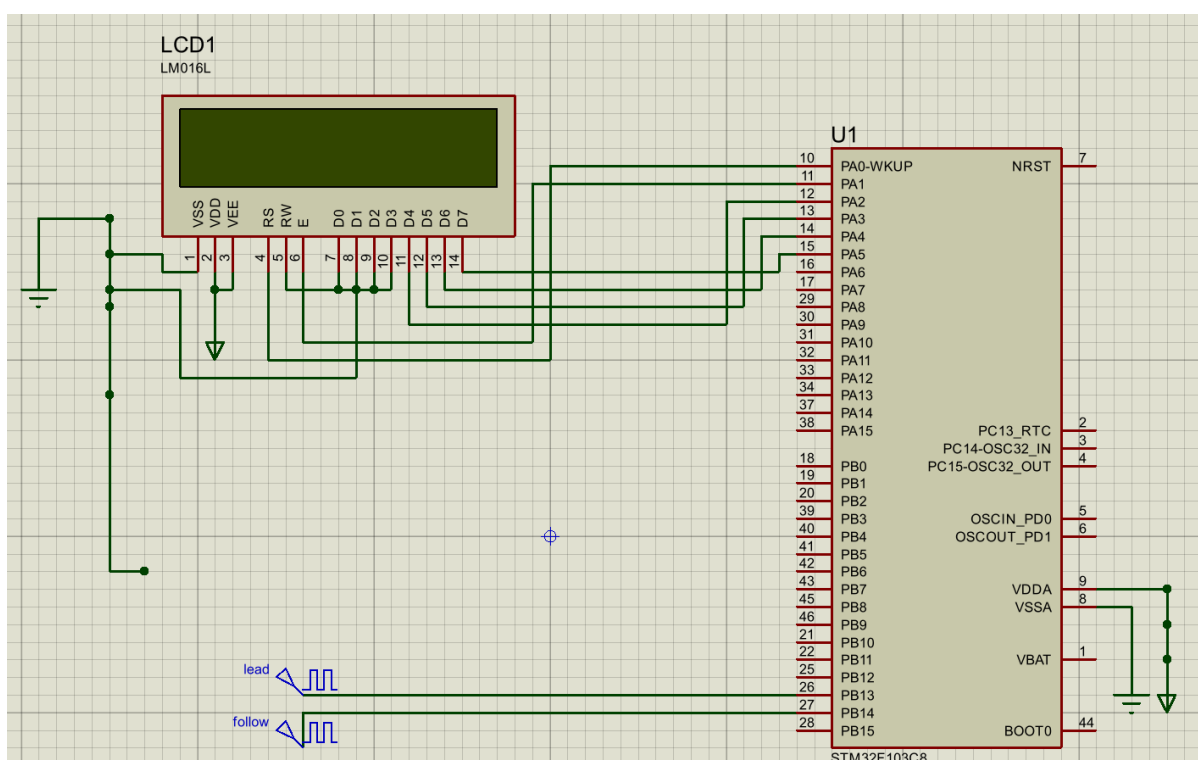
در پایان هر تکرار حلقه، یک تأخیر 200 میلی‌ثانیه‌ای اعمال می‌گردد تا از بارگذاری بیش از حد پردازنده جلوگیری و اطلاعات با نرخ مناسب تازه‌سازی شوند.

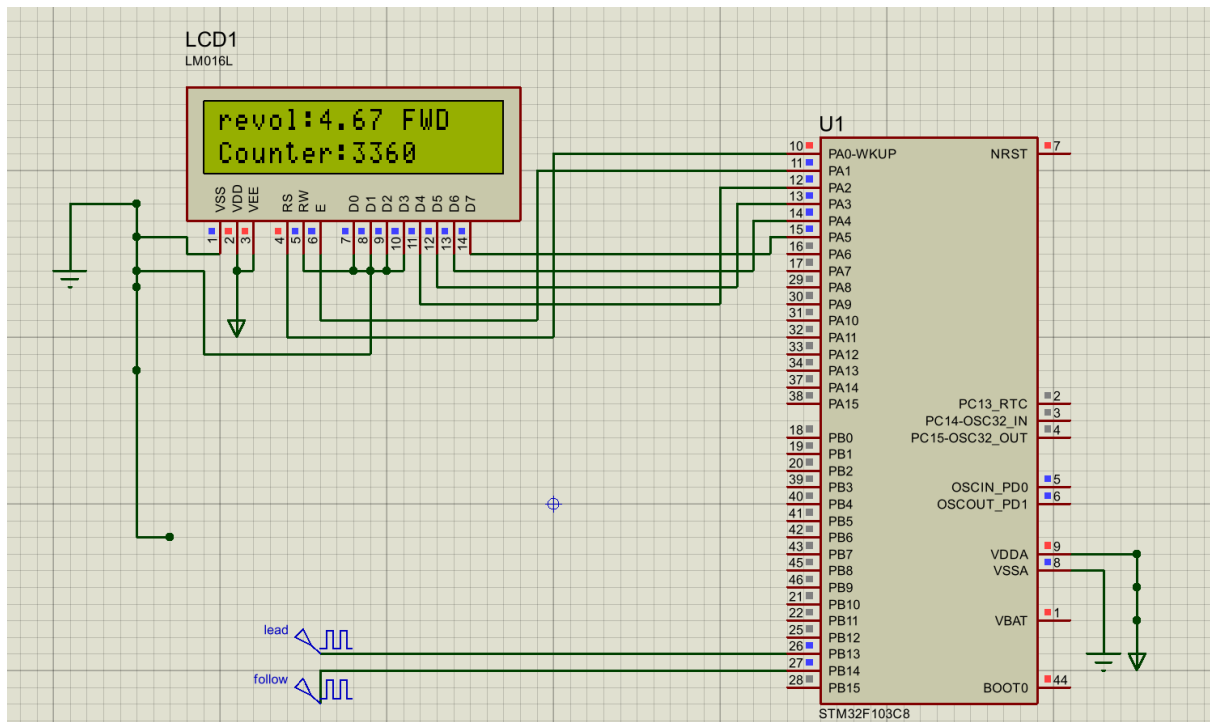
(iii) بخش شبیه سازی Proteus

مدار را مانند شکل زیر می بندیم.

میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) و ژنراتور ها با اختلاف زمانی 250 میکرو ثانیه و LM016L LCD رو در محیط شبیه سازی قرار میدهیم و مدار را مانند شکل زیر می بندیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.





ویدیوی مربوط به شبیه سازی در داخل پوشه تمرین مورد نظر موجود می باشد.